Еремин Олег Юрьевич

	Состояние позиции
00	Нет шашки
01	Шашка красная

pos_1 = (триг. игрока) = player_red =
pos_0 = (триг. игрока) = player_red
CD = correct_to

Операционные системы

- 17 лекций
- РЗ 7к3

#"У нас есть только малышки и девочки, всех промежуточных форм низших у нас нет."

- 4 ЛР
- 1 ДЗ

История развития ОС

05.09.24 ЛЕК

Операционная система — это система, которая работает одновременно и с человеком, и с устройствами, и с приложениями

#"Выпускник ВШЭ с факультета Тик-Тока"

#"С соседями нужно иметь дружбу. Не будьте жлобы дружбу — квартиру обменят"

#"Люди шахидского вида"

Существует 5 поколений ЭВМ:

1. Механические
2. Ламповые
3. Транзисторные

	Название этапа	Сост. трич. пр.	Сост. трич. вых.
1.	Выбор ОТКУДА 1-го игрока	0	0
2.	Выбор куда...	0	1

4. Интегральные микросхемы
5. Микропроцессоры (Сверх Большие Интегральные Схемы)

Появляется ОС MULTICS.
Создан UNIX. — 1972 г. — язык C.
Все принципы за 50 лет не изменились!
80-е годы — IBM PC — CP/M ос для IBM PC
Билл Гейтс купил 86-DOS → MS-DOS
UNIX → BSD → Free BSD (фрахр) → Darwin → MacOS/IOS
UNIX → GNU
↓
MINIX
↓
ядро LINUX ⇒ Android
GNU/Linux
Отвечаем скоробой стилем Windows.
Захотели разработать OS/2 (полусь) ⇒ Windows 95 ⇒
⇒ Windows NT
95 → 98 → ME 😐
NT → 2000 → XP → ... → 11.

	Состояние позиции
00	Нет шашки
01	Шашка красная

pos_1 = (триг. игрока) = player_red
pos_0 = (триг. игрока) = player_red
CD = correct_to

Классификация, основные понятия и характеристики ОС

12.09.24 ЛЕК

- Основные принципы построения ОС:
  - > Частотный принцип — чаще = ближе
  - > Принцип модульности — объединение схожих частей
  - > Принцип функциональной избыточности — 20 на 80
  - > Принцип генерируемости — машин много, система одна
  - > Принцип функц. избыточности — то что нужно и чуть-чуть
  - > Принцип "по умолчанию" — делаешь разумный минимум
  - > Принцип перемещаемости — не привяз. к конкр. месту
  - > Принцип защиты — защита от пользователя
  - > Принцип независимости программ от вн. устройств
  - > Принцип открытой и наращиваемой ОС
- Средства, механизмы, классификация ОС:
  - Система управления процессами
  - Система распределения ресурса
  - Система распределения оперативной памяти
  - Подсистема управления вв/вы-вывода
  - Система управления данными

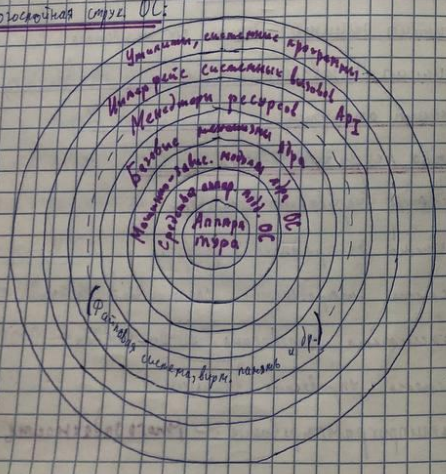
#Мультипрограммирование ~ многозадачность

#

• Типы ОС:
> Универсальные ОС
> ОС специального назначения:
- ОС реального времени (жёсткого р.в. / мягкого р.в.)
- сетевые ОС
- защищённые системы

"Бабушка не мусор – ей ещё жить и жить"
"Database basis"

• Многослойная структ. ОС:



• Типовые средства аппар. подд. ОС:
- средства поддержки привилегированного режима
- средства трансляции адресов
- средства переключения процессов
- система прерываний
- системный таймер
- средства защиты областей памяти

• Функциональные компоненты ОС:
- управление процессами
- управление памятью
- управление файлами и внешними устройствами
- защита данных и администрирование
- интерфейс прикладного программирования
- пользовательский интерфейс

Генерация и переносимость ОС.

Инсталляция – лучший момент для настройки ОС под конкретную вычислительную машину.
Загрузка – неплохой момент для настройки ОС под машину.
POST – Power-On Self Test
BIOS/EFI/UEFI – базовая конфигурация

• 2 принципа построения ядра ОС:
- монолитное ядро (Linux)
- микроядро

Другие подходы:
~ гибридное ядро
~ экзоядро
~ нано ядро
~ ваш вариант :)

• Переносимость ОС:
ОС зависит от архитектур процессора и вычислительной системы.
Первый пример переносимости – UNIX (на C с ещё разным компилятором). Также переносимости поспособствовало создание API и создание абстракций

• Аппаратно-зависимые компоненты ОС:
- средства трансляции адресов
- средства защиты областей памяти
- системы прерываний
- системный таймер
- средства переключения процессов
- средства поддержки привилегированного режима

• Архитектуры процессоров:
> гарвардская
> стэнфордская

Вычислительная система

Вычислительная система – это совокупность взаимосвязанных и взаимодействующих процессоров или ЭВМ, периферийного оборудования и программного обеспечения, предназначенную для сбора, хранения, обработки и распределения информации.

• Типы ВС:
1. Однопрограммные и многопрограммные
2. Индивидуального и коллективного пользования
3. С разделением времени и с пакетной обработкой
4. Однопрог., многопрог. и многомашинные
5. Сосредоточенные и распределённые
6. Работающие и не работ. в режиме реал. времени

7. Универсальные, специализированные и проблемно-ориентированные

• Режимы работы ВС:

1) Мультипрограммирование
2) Режим реального времени
3) Однопрограммный
4) Многопрограммный
5) Пакетная обработка
6) Конкурентный доступ

• Виды ресурсов ВС:

Под ресурсом понимают какой-либо объект, который может распределяться внутри ВС между конкурирующими за него процессами.

Ресурс выделяется процессу на определённый интервал времени. Ресурсы запрашиваются, используются и освобождаются процессами.

> По форме реализации:
- аппаратные (Hard)
- программные (Soft)
- информационные

> По способу выделения:
- неделимые
- делимые

> По реальности существования:
- физические
- виртуальные

> По месту размещения:
- локальные
- удалённые

Процессы

26.09 лек

Процесс — это работающая программа

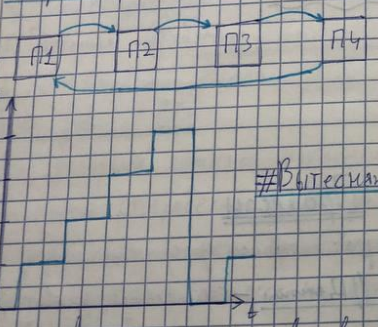
Сервис / Служба // Демоны — системные фоновые процессы

#Всё время выполняется 1 процесс, при этом система не перестаёт быть многозадачной

• Атрибуты процесса:
- память
- процессорное время
- счётчик команд
- регистры
- стек
- идентификаторы

• Виды буферной памяти:
> КЭШ-память
> Оперативная

• Изобразим работу процессов/задач:



#Вытесняющая многозадачность

• Порядок выполнения процессов зависит от процедур распределения процессов.

#"Пишем нашего уровня люди, то есть криворукие"

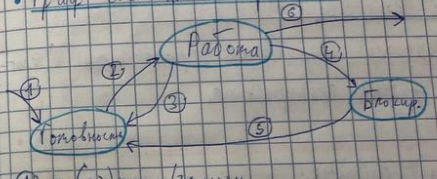
• Проблема вып. мног. заключается в том, что мы не можем определить t конца процесса.

fork() — системный вызов форк.

• Иерархическая структура процесса:
> Родительские процессы
> Дочерние процессы
> Группы процессов

#Первый процесс: init // systemd

• Граф состояний процесса:



1 — Создание / запуск
2 — Исполнение процесса
3 — Конец процессорного времени
4 — Проблемы с ресурсами
5 — Наличие ресурсов
6 — Завершение

• Создание процессов:
1) Инициализация системы (init);
2) Системный вызов fork();

3) Вызов пользователя;
4) Инициация пакетного задания.

• Завершение процессов:
1. Конец программы;
2. Завершение из-за ошибки;
3. Выход по критич. ошибке;
4. Уничтожение другим процессом.

управляемые программистом (процессом)
неуправляемые

• Планирование процессов:
#Передаёт процессор управление процессами в соответствие с алгоритмом планирования.

Среды:
1) Интерактивные системы;
2) Системы реального времени;
3) Системы пакетной обработки данных.

#ОС содержит таблицу процессов.

• Таблица процессов:
- состояния процесса;
- распред. памяти;
- регистры;
- счётчик команд;
- указатель стека;
- указания на занятые ресурсы;
- ...

Алгоритмы планирования процессов

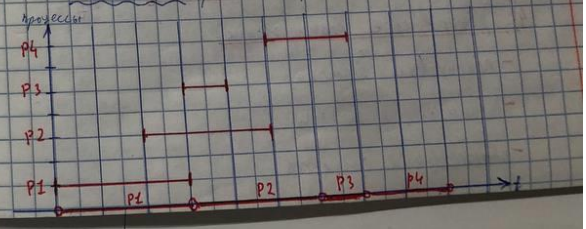
03.10.24
ПСК

• Показатели эффективности выч. систем:
1. Пропускная способность.
2. Оборотное время
3. Коэффициент использования процессора.

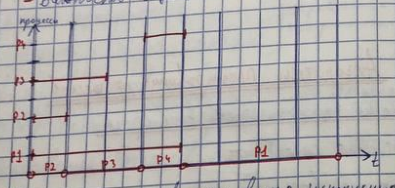
• Алгоритмы планирования процессов:

I Пакетные системы:

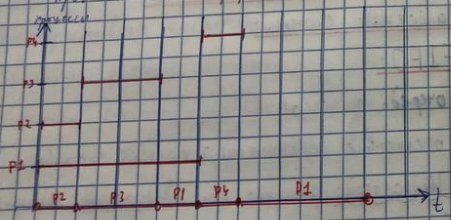
1) FIFO:
- очередь процессов
- отсутствие переключений
- Недостаток: различия при длительности планирования



2) "Кратчайшая задача — первая"
- невытесняющий алгоритм
- важность ожидания



3) "Наименьшее оставшееся время исполнения"
- вытесняющая многозадачность
- процессы могут прерываться



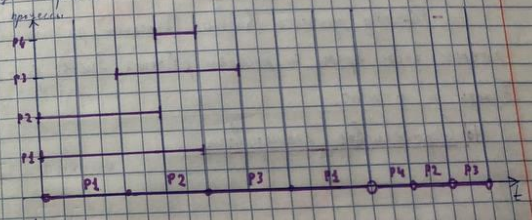
4) "Трёхуровневое планирование"
- планировщик доступа
- планировщик памяти
- планировщик процессов

Планирование в интерактивных системах

10.10.24
ПСК

II Интерактивные системы:

5) Циклическое планирование
- выделение кванта времени
- по окончании кванта управление передаётся следующему процессу, а текущий процесс становится в конец очереди.



$t_k = 2$

6) Приоритетное планирование
- очередь процессов с разными приоритетами.
- PR (priority), NI (nice)
- SU (superuser), SUDO (substitute user and do)
- внутри одного приоритета циклическое планирование
- приоритеты бывают статические и динамические.
- "инверсия ДКА"

7) "Самый короткий следующий"

– аналог алгоритма 2)

8) Гарантированное планирование

– в реальной жизни не используется

– в системе n процессов, каждому процессу гарантируется $\frac{1}{n}$ времени.

9) Лотерейное планирование

– Каждому процессу выданы "билетики", разыгрываем лотерею, даём время и пускаем работать. Заново раздаём билеты и по новой.

10) Справедливое планирование

Потоки

0.24

- У процесса свой адресное пр-ство

Бывают ситуации множественного взаимодействия в общем адресном пр-ве

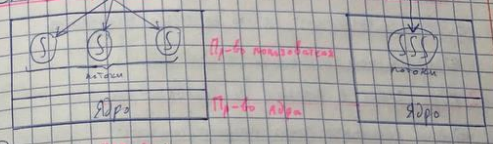
- Способы задания параллельных систем:

1. У каждого своя память
2. Общая память

} 3. И то, и то.

Поток (S) – часть процесса

- Потоки имеют общее адресное пр-во.
- Схема процессов:



- Ресурсы потока:
  - счётчик команд
  - регистры
  - СТЭК
  - состояния
- Общие (разделяемые) ресурсы потоков:
  - адресное пр-во
  - глоб. перем.
  - открытые файлы
  - дочерние процессы
  - информация об исп. ресурсах
  - сигналы и их обработчики

Потоки чаще всего не имеют иерархию.

Тр. ист. = Тр. ц

Поток может занять весь квант времени процесса

- Принципы построения многопоточных систем:

1. Ядро не знает о наличии потоков
   - ⊖ работа по переключению потоков на плечо программиста
   - ⊕ ядро не надо модифицировать
2. Ядро знает о потоках и управляет ими
   - ⊕ легко переключаться между потоками, управлять
   - ⊖ модиф. ядро
3. Ядро знает, но не управляет
   - ⊕ у программиста больше власти, ядро меньше разр. время
   - ⊖ модиф. ядра + больше работы программисту

Найдите возможность не пользоваться потоками

- Проблемы потоков:

> Процесс вызывает дочерний процесс, вопрос: будет ли там столько же потоков?

> Блокируют ли потоки родителей потоки потомков?

> Если пришли данные, все ли потоки их получают?

> Если один поток закрыл файл, а другой хочет его использовать, что делать?

> Потоки могут быть не вкурсе о выделении памяти.

"Кто-то решил нажаться своими программами на кнопках"

Межпроцессовое взаимодействие

17.10.24 лек 2.0

IPC – inter process communication

Процессы должны обмениваться информацией. Процессы могут очень тесно и много взаимодействовать.

LPC – local procedure call

RPC – remote procedure call

- Проблемы межпроц. связей:
  1. Передача информации;
  2. Ситуация попадания двух процессов в крит. (критическую) ситуацию;
  3. Согласование действий процессов.

"Слава Богу не на мне читаю"

24.10.

- Ситуация, в которой 2 или более процессов считывают информацию одновременно, конечный результат зависит от того, какой был первым, называется состояние состязания
- Очевидным способом избежать состояния состязания является запрет чтения данных разделяемых данных

Тр. ист. = Тр. ц

разными процессами (взаимное исключение)

• Часть программы, где есть обращение к общим ресурсам называется критической секцией.

Условия избежания состязаний:

1. 2 процесса не должны одновр. нах. в крит. секции.
2. В программе не должно быть предположений о скорости и количестве процессов.
3. Процесс вне крит. секции не может блокировать процессы.
4. Невозможна ситуация бесп. ожид. в крит. области.

"Втолка, сшивающая ся — это верх человеческой мысли"

• Способы обеспечения взаимоисключений:

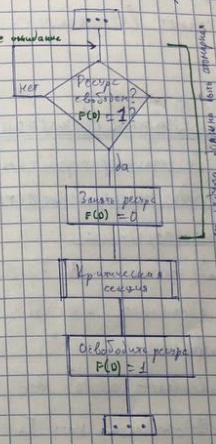
1) Операционная система разрешает процессу запрещать прерывания на время его нахождения в кр.с.
2) Использование блокирующих переменных.
3) Аппаратные средства (неделимые команды процессора)
4) Передача сообщений
5) Барьеры

Блокирующие переменные:

Все процессы имеют к ним доступ.

P — ресурс

F(P) — блокирующая переменная (mutex)



Mutex — mutual excertion. Всегда бинарная

Семафоры:

~ Семафоры Дейкстра

Используются целые неотрицательные переменные и два примитива.

$S \in Z$

P и V: V(s) — S++ ; P(s) — S-- если это возможно

Если S = 0, то процесс, вызвавший P(S), ждёт, пока это уменьшение не станет возможным.

Семафоры Дейкстры. Мониторы

"Кто боится до мистериумов, тем рассказ про дом без правил сроки несовершеннолетних"

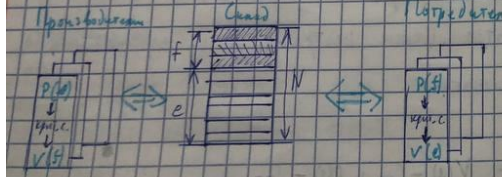
• Задача "Производители — потребители":

— буферный пул (склад) состоит из N буферов

— произв. и потреб. обращаются к складу с разными скоростью и интенсивностью

— произв. приостанавливается, когда все буферы заняты, и активируется при освобождении хотя бы одного.

— потреб. приостан., когда все буферы пусты, и актив. при наполнении хотя бы одного.



Два семафора:

e — кол-во пустых буферов

f — кол-во полных буферов

Начальное состояние:

e = N

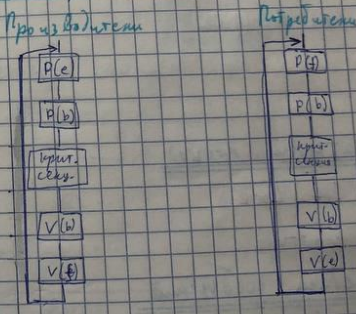
f = 0

Если нет ограничений на количество заказов, то проблем нет.

Проблемы начинаются при вводе мьютекса на наличие кого-то на складе

b — мьютекс — двоичный семафор — разрешение на вход

Тогда алгоритм примет вид:



deadlock, клинч, тупик.

Мониторы

• **Монитор** — это набор процедур, переменных и других структур данных объединённых в особый модуль или пакет. Процессы могут вызывать процедуры монитора, но в процедур вне монитора нет прямого доступа к внутр. структурам данных монитора.

Пример:

```
monitor example
    integer k;
    condition k;
    procedure producer()
    ...
    end;
    procedure consumer()
    ...
    end;
end monitor.
```

Аппаратный подход

TSL — Test and Set Lock

Отрубает шину памяти

Передача сообщений

Чаще исп. в распределённых системах

• Реализуется через два сообщения:

send (destination, message)

recieve (source, message) # может получать от всех

• Подходы к ожиданию сообщений:

- ожидание (блокировка) до recieve
- ожидание по таймеру

• Проблемы:

⊖ потеря сообщений

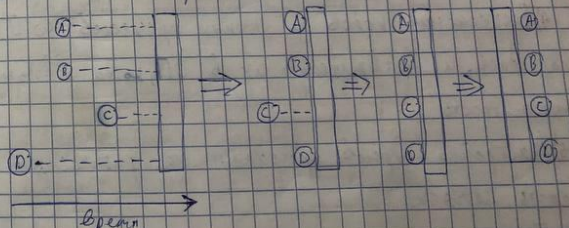
⊖ аутентификация

i2p — безопасное Tor'а

"По интересности мы заняли прокладку по скибиди туалетам"

Барьеры

Механизм синхронизации



Встречается в параллельных системах.

Классические задачи в IPC

1) Задача "Произв. — потреб."
2) Задача "Писатели — читатели"
3) Задача "Обедающих философов"
4) Задача "Спящего брадобрея"

27.11.24

Управление памятью

"Если вы в ратках, стоит бояться косяков"

Под памятью будем понимать оперативную память

• Функции системы управления памятью:

1) Отслеживание свободной и занятой памяти.
2) Выделение памяти процессам
3) Освобождение памяти по завершению процесса
4) Вытеснение кодов и данных процессов из ОП на диск, и возвращение их в ОП, когда освободится место
5) Настройка адресов программы на физическую область памяти.
6) Дефрагментация (сжатие) памяти.
7) Резервирование памяти для служебных информационных структур. (буферная память, блок. пересылки, таблица процессов и т.д.)
8) Защита памяти — запрещение одному процессу читать и писать память другого процесса.

• Типы адресов:

логические, математические адреса

• Модели виртуального адресного пр-ва:

- плоская (flat) [смещение]
- сегментная [номер сегмента; смещение в сегменте]
- другие.

• Способы преобразования в.а. в ф.а.:

1. Перемещающий загрузчик
2. Динамическое преобразование

> Перемещающий загрузчик:

На основании того, что адреса требуют пересчёта, в момент перемещения они пересчитываются

⊕ один пересчёт, более экономично.

⊖ жёсткая привязка к области памяти.

> Динамическое преобразование:

Программа загружается в память в неизменном виде. Преобразование осуществляя при обращении к программе.

⊕ позволяет перемещать программный код при выполнении.

⊖ частый раз пересчёты

Распределение памяти

• Алгоритмы распределения памяти:

– Проблемы:

– Нужно ли выделить непрерывную область памяти?

– Можно ли переносить сегменты памяти?

– Как уместить все сегменты в память?

Методы распределения памяти

Без использования внешней памяти

1. С фиксированными разделами
2. С динамическими разделами
3. С перемещаемыми разделами

С использованием внешней памяти

1) Страничное распр.
2) Сегментное распр.
3) Сегментно-страничное

~ Подробно:

1. Самый примитивный способ. Делим память на разделы и подгружаем в/процессы. Проблема: жёсткие границы.

2. Возможность изменения размера раздела под процесс. Проблема: при наличии памяти не всегда можно загрузить процесс в ОП. Алгоритмы: "Первый подх. подход размера", "Лучш. подх. размер", "Худш. подх. размер"

3. Возможность двигать процессы в памяти. Сжатие. Проблема: когда осущ. операцию сжатия?

Swapping – выгрузка из ОП полностью

Страничная подкачка (Виртуальная память) – выгрузка частями

1) Вся память делится на страницы, в которые загружаются и выгружаются данные. У каждого процесса своё ВАП, со своим распределением на страницы.

Выгрузка в физическую память не ограничена порядком записи. Создаются таблицы, хранящие информацию о нахождении страниц процесса в памяти.

– Дескриптор страницы – содержит номер физической страницы и информацию о странице (признак присутствия, признак модификации, признак обращения к странице)

~ Страничное прерывание – остановка процесса для подгрузки недостающей страницы.

Страницы кратны 2. логично

Внутри страницы линейная адресация

2) Деление на сегменты происходит с учётом смыслового контекста. Сегменты могут быть разного размера. Они тоже выгружаются в память, информация о сегментах так же хранится в таблицах.

Проблема: необходимо фрагментировать память.

3) Берутся сегменты, и сами сегменты делятся на страницы. Создаются сегментные и страничные таблицы. Тем самым увеличиваются вычисления из-за увеличения параметров адресации сегментно-страничного распределения

Подсистема ввода-вывода

• Задачи подсистемы:

1) Организация параллельной работы устройств ввода-вывода и процессора.

Контроллер DMA – Direct Memory Access

2) Согласование скоростей обмена и кэширования данных. (Буферизация или синхронизация)

3) Разделение устройств и данных между процессами. Доступ к данным может быть монопольным (последовательным) и разделённым (параллельным).

4) Обеспечение удобного логического интерфейса между устройствами и остальной частью системы.

5) Поддержка широкой системы драйверов с возможностью включения нового драйвера.

DDI

Kernel

DKI

DDI – Device Driver Interface

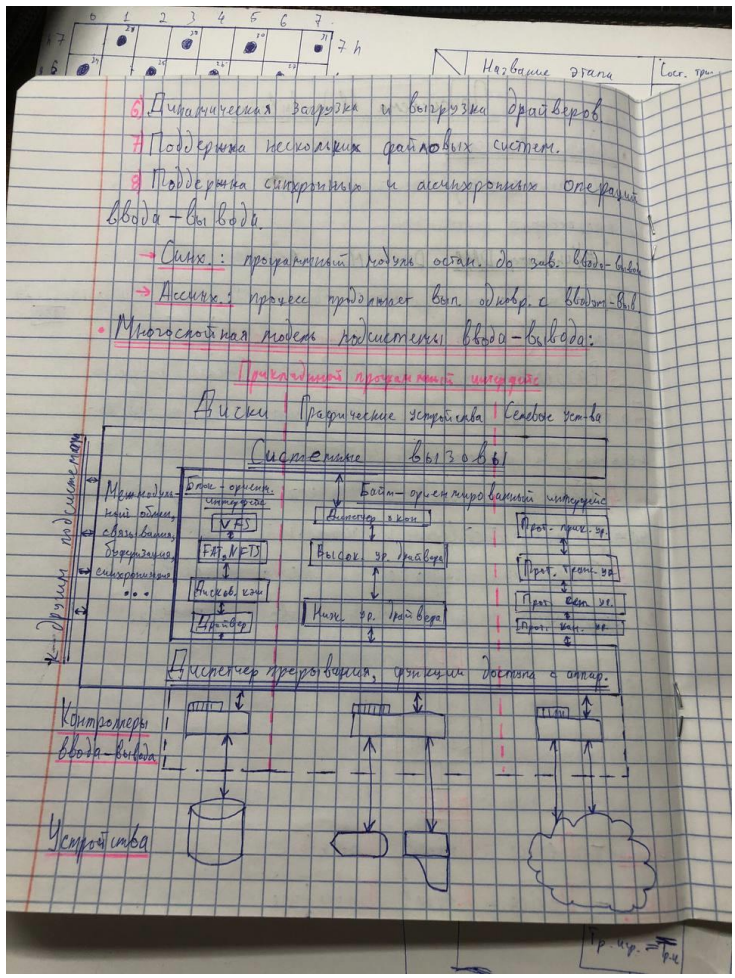
DKI – Device Kernel Interface

6. Динамическая загрузка и выгрузка драйверов.

7. Поддержка нескольких файловых систем.

8. Поддержка синхронных и асинхронных операций ввода-вывода.

→ Синх.: прозрачный модуль остан. до зав. ввода-вывода

→ Асинх.: процесс продолжает вып. одновр. с вводом-выв.

• Многослойная модель подсистемы ввода-вывода: